

CC5502-1 Geometría Computacional**Profesor:** Nancy Hitshfeld K.**Ayudantes:** Catalina Gajardo y Gustavo Medel**Estudiante:** Andrés Calderón

Tarea 2 y 3

Cerradura Convexa

1. Experimentación

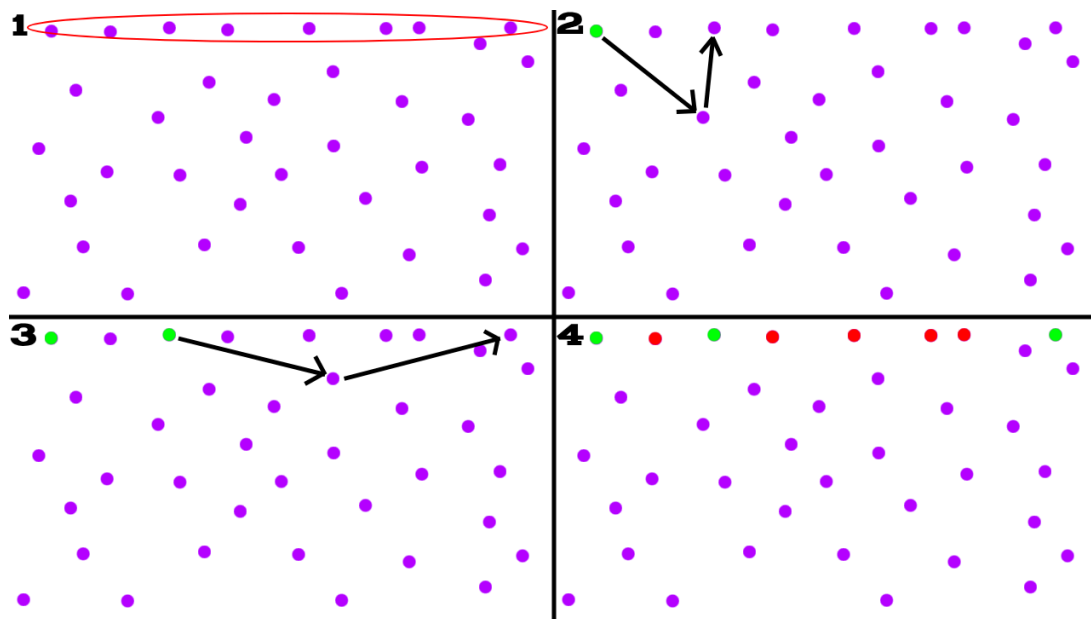
Se utilizó como sistema operativo Arch Linux, con la siguiente CPU: Intel(R) Core(TM) i3-8130U @ 3.40 GHz y 4 GB de RAM, por temas de tiempos de ejecución se decidió no ir más allá de 10^5 puntos.

Se utilizaron 3 clases distintas para la generación de los distintos casos de prueba:

1. Uno genera puntos aleatorios dentro del área de un cuadrado.
2. A otro se le indica un porcentaje para formar la cantidad de puntos que estarán en una circunferencia, mientras que el resto de puntos se encontrarán dentro de este borde definido.
3. Por último, un tercer método que realiza lo mismo que el algoritmo anterior pero con un cuadrado en vez de un círculo, con el fin de tener una gran cantidad de puntos colineales y se utilizó en particular para la comparación de tiempos al usar enteros contra números reales.

2. Implementación

El algoritmo de Gift Wrapping implementado como se sabe es sensible a la salida ya que su tiempo de ejecución depende de la cantidad de puntos en la cerradura convexa, pero para el caso particular donde muchos de los puntos en esta cerradura son colineales el algoritmo pasa a ser mayormente sensible a la entrada, es algo difícil de explicar, así que usaré la siguiente imagen para que sea más entendible:



Imaginar que los puntos morados de cada cuadrante son el mismo conjunto de puntos en distintos momentos solamente, en este caso según el paso 1 los puntos dentro de la línea roja son colineales y parte de la cerradura convexa. Considerar que en este caso se hace *wrapping* en sentido CW en vez de CCW

En el paso 2 se determina el punto más a la izquierda siendo este verde, y luego meramente por el orden en que venían los datos encontró el siguiente punto de la cerradura convexa, pero notar que solo por el orden en que apareció es que se saltó un punto colineal intermedio, lo mismo pasa en el punto 3, finalmente generando que se salten todos los puntos rojos mostrados en el paso 4.

Con esto se quiere ilustrar que la implementación para el caso donde hay muchos puntos colineales en la cerradura convexa tan solo incluye una cantidad incierta de puntos totalmente dependiente del orden en que vengan dados los puntos.

Intenté hacer que fuera consistente con el segundo algoritmo que decidí implementar, el cual fue incremental ya que este no incluye puntos colineales, pero no lo logré por distintas razones, ya sea porque no lograba terminar el algoritmo por quedarse *loopeando* en casos borde, o por el contrario se incluían menos puntos de los que realmente hay por acumulación de errores, a veces llegando a casos borde como generar una cerradura de tan solo 2 puntos, lo cual es imposible para estos casos.

Cabe mencionar que los algoritmos efectivamente entregan la misma cerradura convexa, con la única diferencia que si existen puntos colineales entonces el algoritmo de Gift Wrapping puede incluir puntos extra en comparación al incremental que no los añade.

3. Análisis

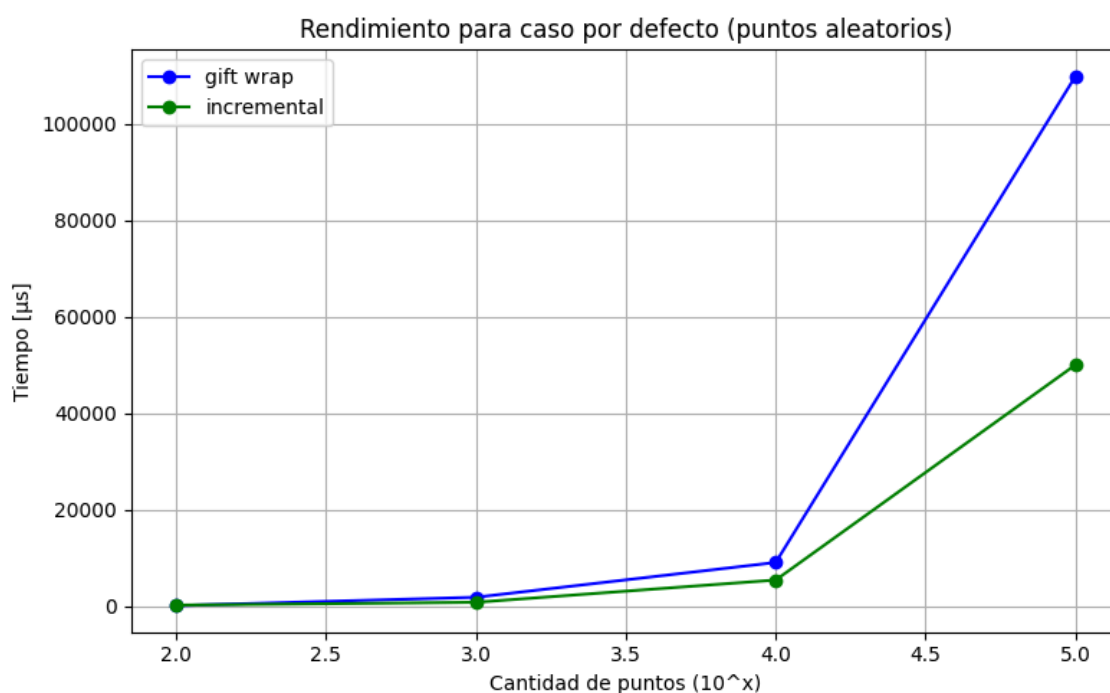
Como se mencionó en el punto anterior, se tiene una cantidad aleatoria de puntos en la cerradura convexa según la cantidad de puntos colineales que existan, por esto mismo los tiempos no serán tal y como se esperan para algunos casos.

Esta cantidad se puede revisar en el csv generado al ejecutar la tarea pero no se expondrán explícitamente por la dificultad de mostrarlo junto a todos los demás datos.

Lo más importante es que para el caso en que se generan puntos en un círculo el algoritmo de Gift Wrapping si es capaz de hallar todos los puntos esperados en la cerradura, mientras que en el resto de casos, al ser cuadrados, independiente de si se tienen puntos colineales o no, el algoritmo no suele pasar un total de 50 puntos en el resultado final, de esta forma, mostrando una diferencia de ejecución de tiempo exagerada entre estos.

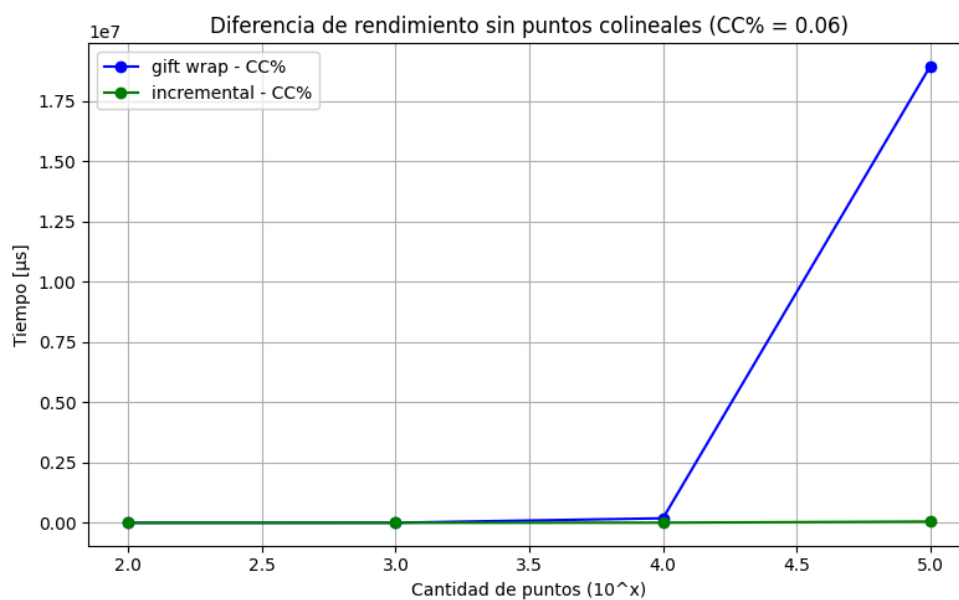
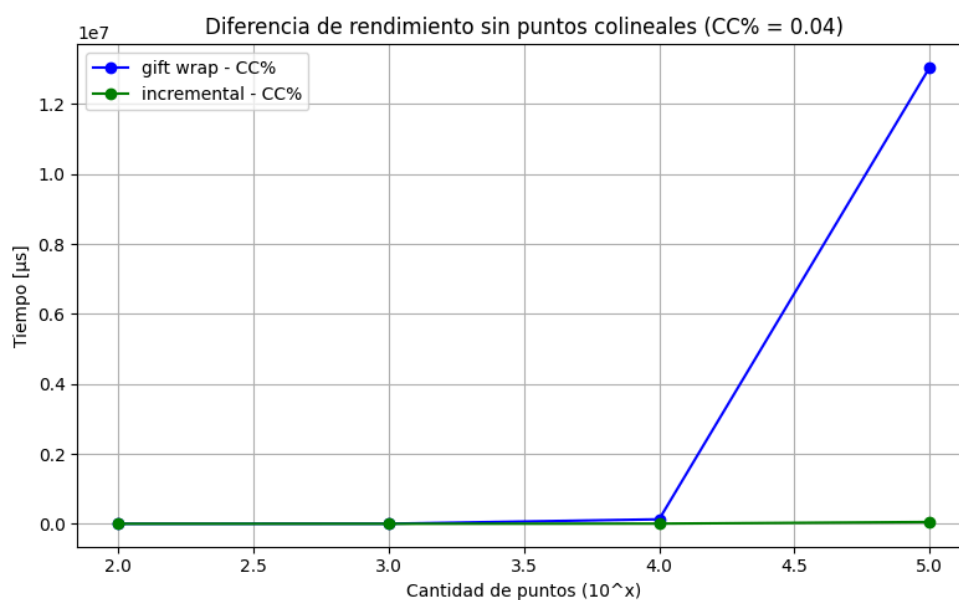
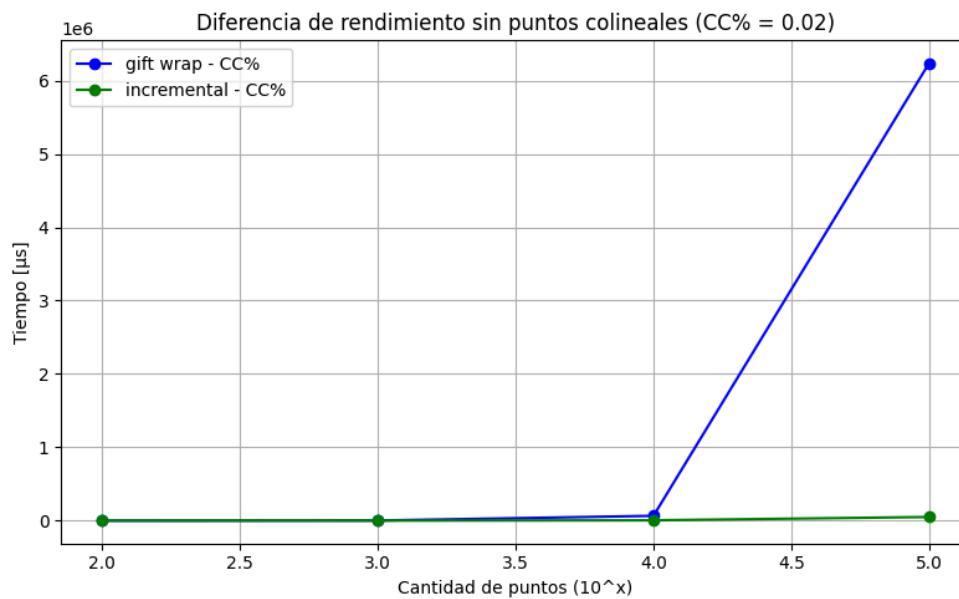
A continuación se exponen los distintos resultados obtenidos para la experimentación particular que se realizó en esta tarea:

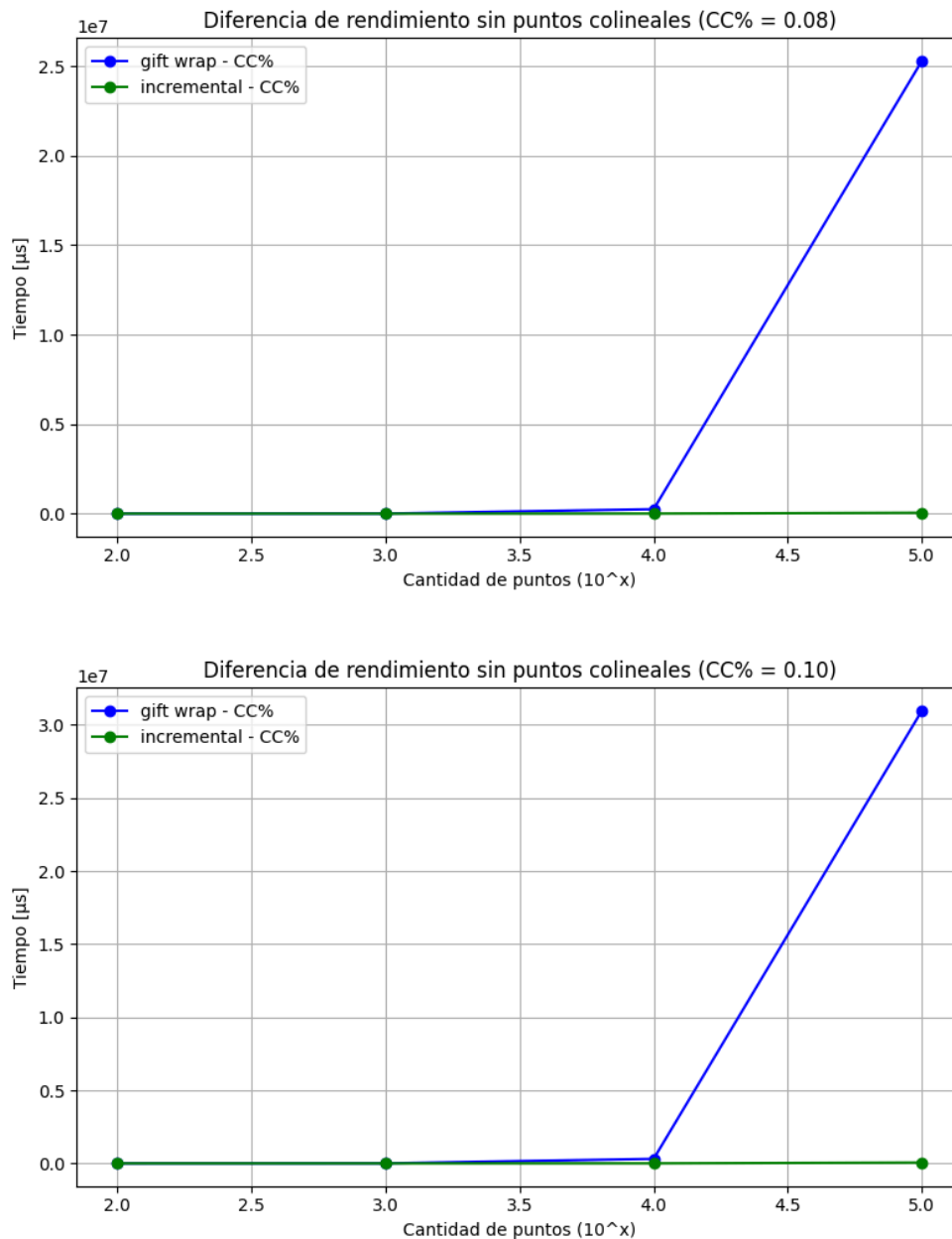
3.1. Caso por defecto (puntos aleatorios)



Con esto podemos determinar que el algoritmo incremental es mejor para puntos aleatorios

3.2. Caso cierto porcentaje en la cerradura convexa



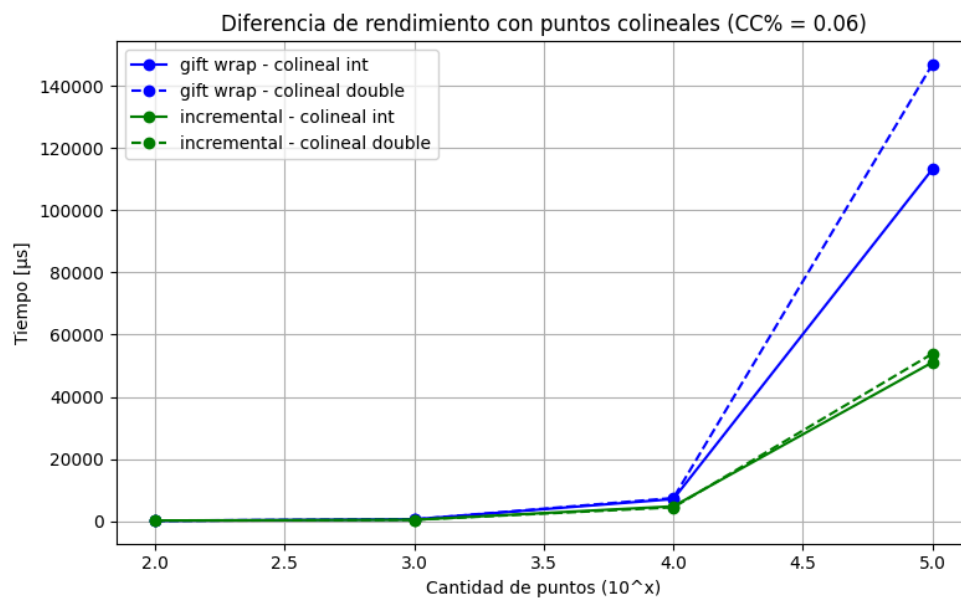
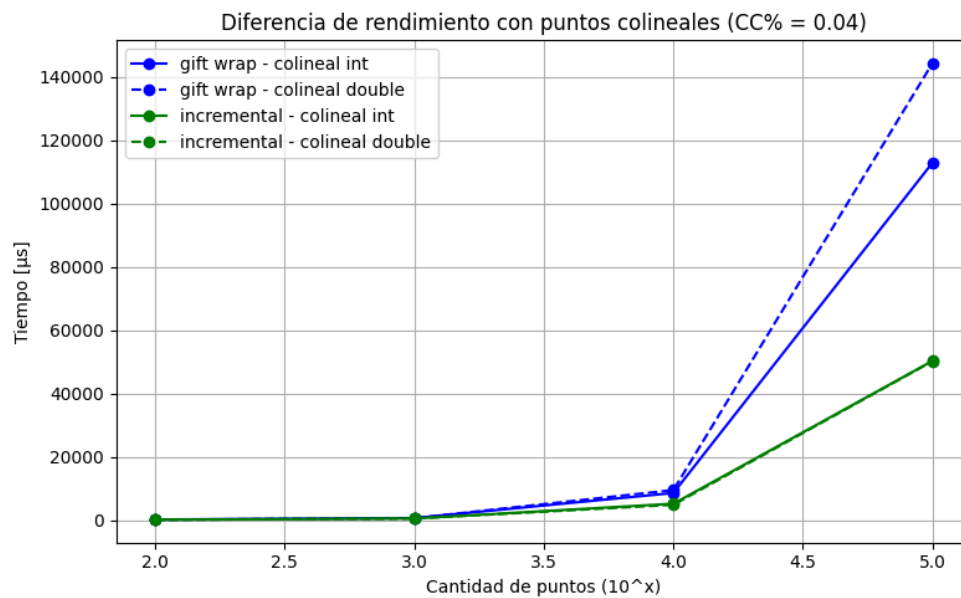
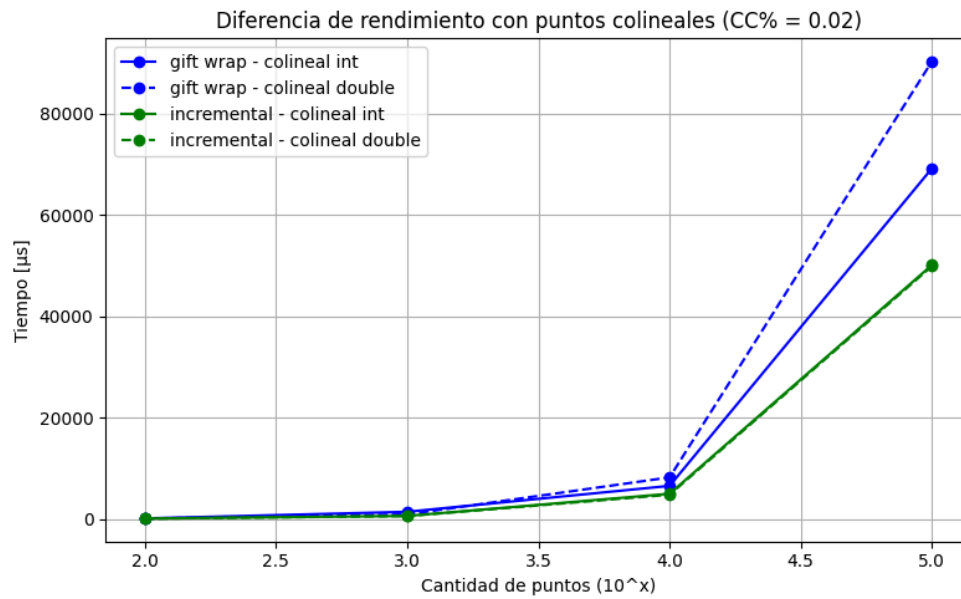


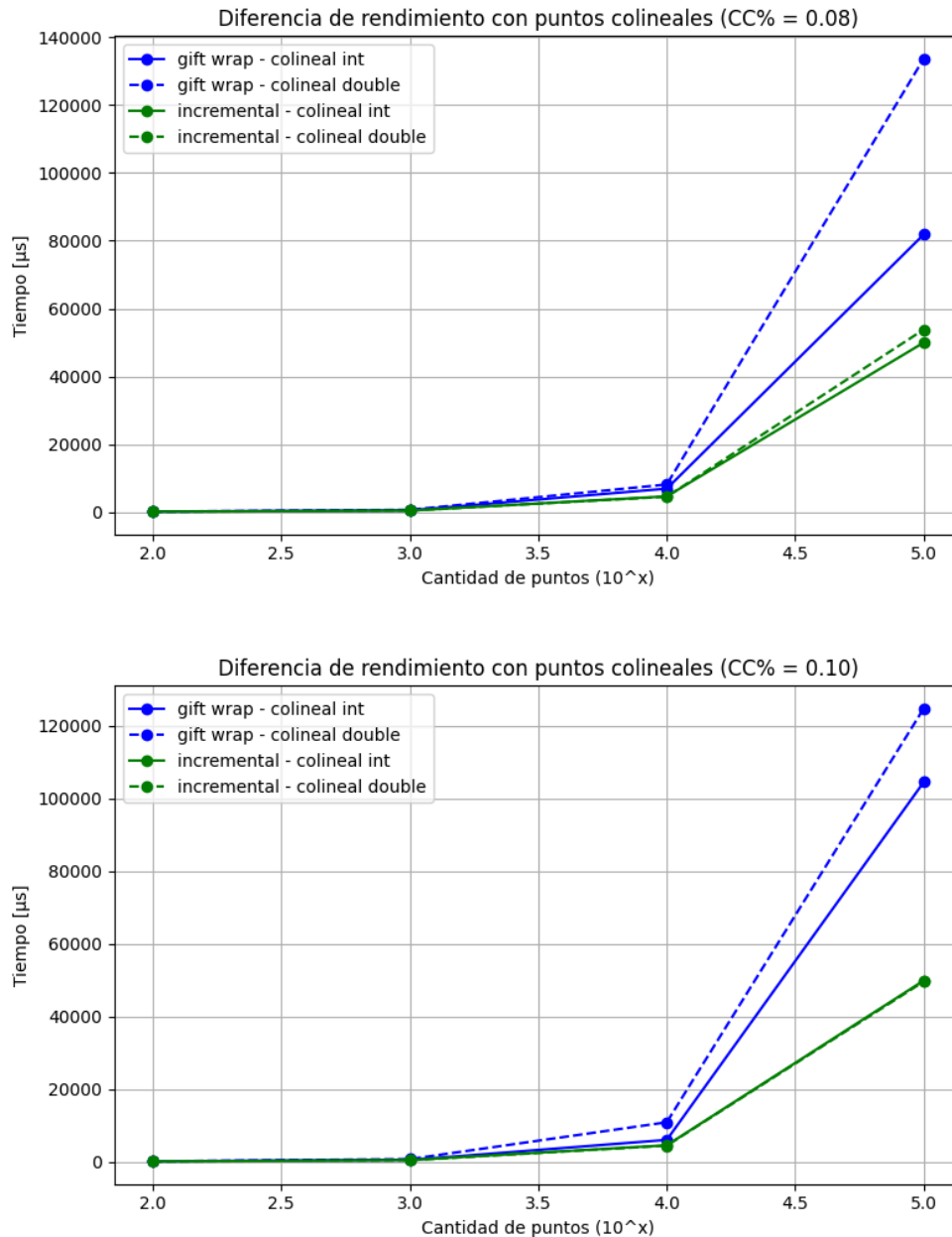
La principal diferencia que se puede apreciar entre estas iteraciones radica solamente en que a medida que el porcentaje aumenta, el algoritmo de Gift Wrapping se demora más, esto debido a que el tiempo de ejecución de este depende de la cantidad de puntos que estén en la cerradura convexa.

Considerando que para este caso se utilizó un círculo, el algoritmo encuentra todos los puntos que están en la cerradura, y por ello es que la diferencia es tan notable a medida que el porcentaje incrementa, de esta forma, aún si el porcentaje es tan pequeño como 2%, el algoritmo incremental sigue siendo más eficiente.

Es posible determinar el porcentaje real de puntos que pertenecen a la cerradura convexa respecto al total en casos no controlados mirando solamente a la cantidad de puntos que se retornan por el algoritmo, en particular usando el incremental para excluir los colineales.

3.3. Caso puntos colineales para comparar int con double





Finalmente, podemos hacer una comparativa entre los tiempos de ejecución cuando se utilizan números enteros contra reales, para el caso particular en que el se tienen muchos que son colineales en la cerradura convexa, y de esto podemos evidenciar que por lo general utilizar double por sobre int ralentiza la ejecución ligeramente, pero no lo suficiente como para cambiar el orden esperado teóricamente.

4. Conclusiones

Los tiempos de ejecución no es muy posible verificarlos de forma segura dados los tamaños que se pudieron ejecutar, pero al menos es posible decir que el incremental es mucho más eficiente que el algoritmo de Gift Wrapping, lo cual se condice con la teoría.

Y también que este último es dependiente de la cantidad de elementos que contenga la cerradura convexa, ya que para el caso del círculo al hallar todos los puntos dentro del porcentaje, cuya cantidad llega hasta 10.000 para el tamaño de 10^5 con porcentaje 10%, el tiempo de ejecución es notablemente mayor a que cuando halla cantidades mucho menores.